KernelLab — Relatório Técnico

1. Introdução

O KernelLab é um simulador didático de sistemas operacionais desenvolvido para demonstrar conceitos relacionados ao gerenciamento de processos, escalonamento da CPU, gerenciamento de memória virtual e controle de dispositivos de entrada e saída.

O projeto utiliza cenários de processos com diferentes tempos de chegada, durações, prioridades, necessidades de memória e dispositivos associados. A partir desses dados, o simulador permite observar o comportamento de diferentes algoritmos e comparar seus resultados.

2. Objetivos

O objetivo geral é desenvolver um simulador capaz de representar aspectos fundamentais do funcionamento de um sistema operacional multitarefa.

Os objetivos específicos são:

- Implementar algoritmos de escalonamento de CPU.
- Calcular métricas de desempenho dos processos.
- Simular algoritmos de substituição de páginas.
- Representar o controle de acesso a dispositivos.
- Gerar gráficos para facilitar a análise dos resultados.
- Comparar os resultados e discutir vantagens e limitações das abordagens utilizadas.

3. Tecnologias utilizadas

O projeto foi desenvolvido em Python. A biblioteca Matplotlib é utilizada para gerar gráficos dos resultados das simulações. Os dados dos processos são armazenados em um arquivo JSON, e os testes automatizados utilizam o módulo `unittest`.

4. Organização do projeto

O projeto está organizado em módulos com responsabilidades específicas:

- `src/main.py`: ponto de entrada e execução das simulações.
- `src/schedulers.py`: algoritmos de escalonamento e cálculo de métricas da CPU.
- `src/memory.py`: simulação de memória virtual e substituição de páginas.
- `src/devices.py`: controle de acesso e filas de dispositivos.
- `src/metrics.py`: geração de gráficos.
- `data/processes.json`: dados dos processos utilizados na simulação.
- `tests/`: testes automatizados.

- `resultados/`: arquivos gráficos gerados pelo simulador.
- `docs/`: documentação técnica do projeto.

5. Metodologia

As simulações utilizam os processos definidos no arquivo JSON e uma sequência de referências a páginas para testar os algoritmos de memória. Os resultados são apresentados no terminal e em gráficos, permitindo comparar o comportamento das técnicas implementadas.

Os resultados quantitativos, a análise dos algoritmos e as limitações da implementação serão apresentados nas próximas seções após a consolidação das execuções e dos testes.

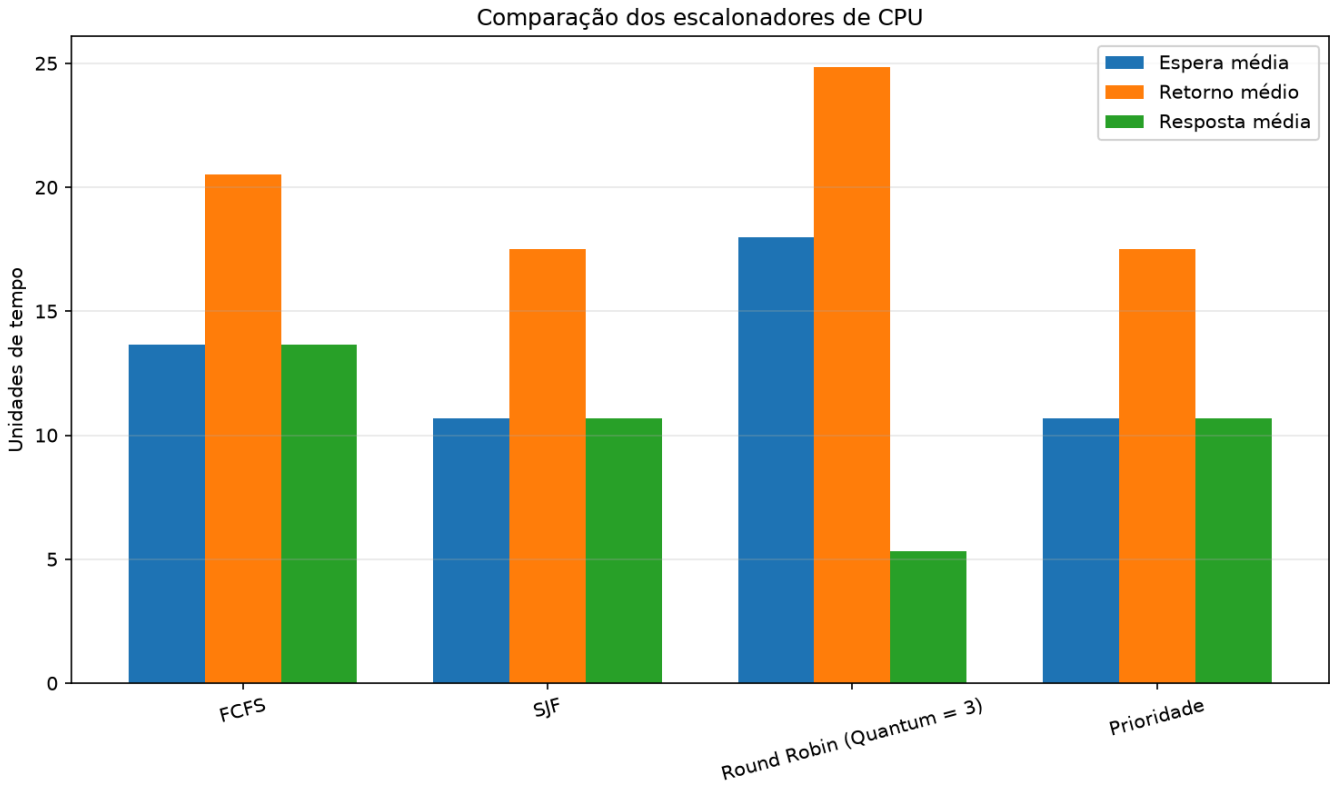
6. Resultados e análise

6.1 Comparação dos algoritmos de escalonamento da CPU

Foram executados quatro algoritmos de escalonamento: FCFS, SJF, Round Robin com quantum de 3 unidades de tempo e Prioridade. A comparação considera o tempo médio de espera, o tempo médio de retorno, o tempo médio de resposta e a quantidade de trocas de contexto.

Algoritmo	Espera média	Retorno médio	Resposta média	Trocas de contexto
FCFS	13,67	20,50	13,67	5
SJF	10,67	17,50	10,67	5
Round Robin	18,00	24,83	5,33	13
Prioridade	10,67	17,50	10,67	5

Figura 1 — Comparação dos algoritmos de escalonamento da CPU



Nesta execução, SJF e Prioridade apresentaram os menores tempos médios de espera e retorno. Já o Round Robin apresentou o menor tempo médio de resposta, mas registrou mais trocas de contexto.

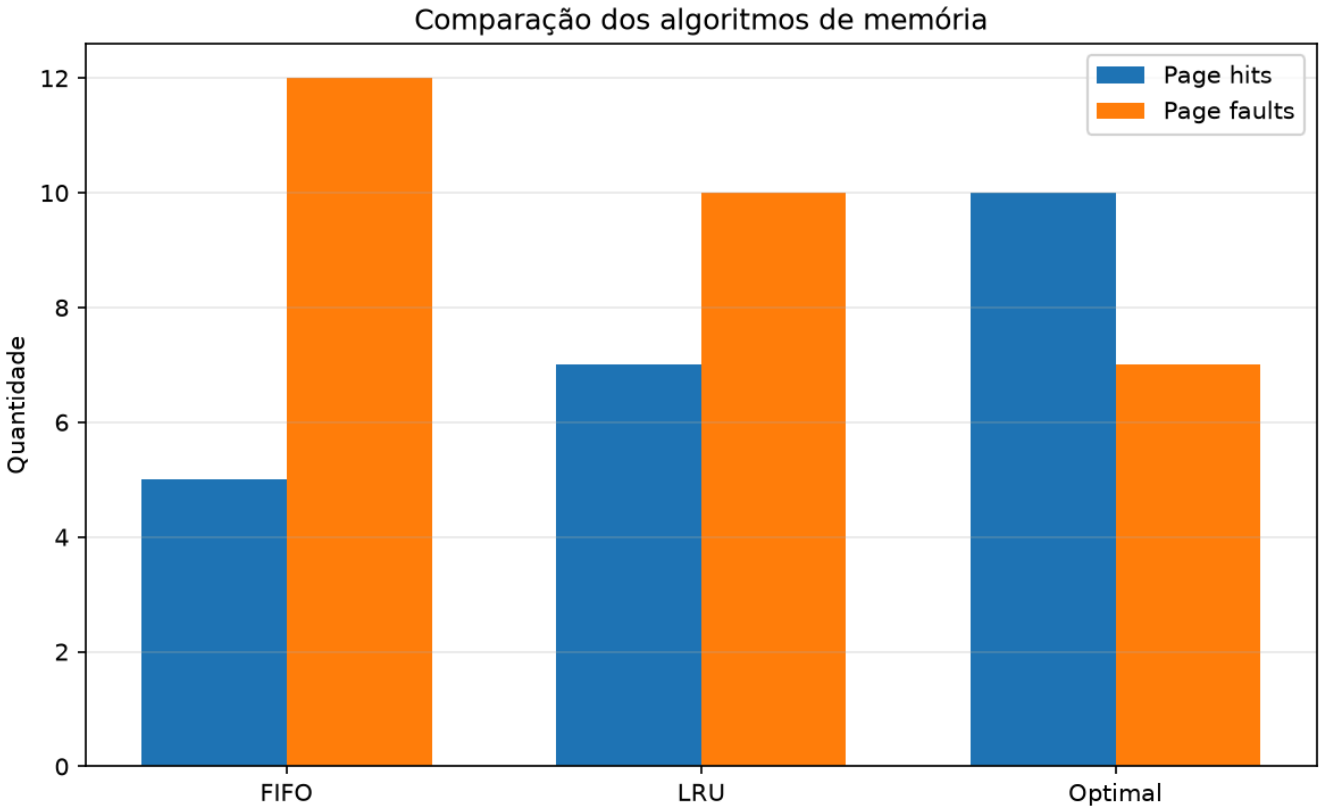
O algoritmo de Prioridade implementado é não preemptivo. Por isso, o processo P4, responsável pela emissão de um alarme crítico, não interrompe o processo P1 quando este já está utilizando a CPU. Para um cenário médico, seria importante avaliar uma estratégia que desse resposta mais rápida às tarefas críticas.

6.2 Gerenciamento de memória virtual

A simulação utilizou quatro frames e uma sequência de 17 referências a páginas.

Algoritmo	Acertos de página	Faltas de página	Taxa de faltas
FIFO	5	12	70,59%
LRU	7	10	58,82%
Optimal	10	7	41,18%

Figura 2 — Comparação dos algoritmos de substituição de páginas



O algoritmo Optimal apresentou o menor número de faltas de página para a sequência utilizada. Isso ocorre porque ele considera qual página será utilizada mais distante no futuro. Como essa informação não costuma estar disponível antecipadamente em um sistema real, o algoritmo serve principalmente como referência de comparação.

O LRU apresentou resultado melhor que o FIFO nesta simulação. Entretanto, os resultados dependem da sequência de referências e da quantidade de frames utilizada.

6.3 Controle de acesso aos dispositivos

O simulador utiliza uma matriz de permissões para controlar o acesso dos processos aos dispositivos. Nos testes realizados, foram aceitas as solicitações de P1 para leitura da rede, P3 para escrita no disco e P4 para uso do áudio. A solicitação de P1 para utilizar o disco foi negada por falta de permissão.

O programa também possui filas de solicitações e mecanismos de bloqueio utilizando `Lock`. Porém, o processamento sequencial das filas não comprova, por si só, uma disputa concorrente real entre threads. É necessário realizar um teste adicional com acessos simultâneos ao mesmo recurso para validar o comportamento da sincronização.

6.4 Ciclo de vida dos processos

Em um sistema operacional multitarefa, um processo pode passar pelos estados de novo, pronto, executando, bloqueado e finalizado. Esses estados representam, respectivamente, a criação do processo, sua espera pela CPU, sua execução, a espera por algum evento ou recurso e a conclusão de suas atividades.

No contexto do KernelLab, os algoritmos de escalonamento representam principalmente a escolha de qual processo utilizará a CPU. A simulação não deve ser interpretada como uma implementação completa de todos os estados e transições de um sistema operacional real.

6.5 Limitações e recomendação técnica

O KernelLab é uma simulação didática com um conjunto definido de processos, uma CPU simulada e uma sequência específica de referências à memória. Portanto, os resultados não representam todos os comportamentos possíveis de um sistema operacional real.

Para o cenário da MedControl Systems, recomenda-se avaliar uma política de prioridade capaz de atender com rapidez os alarmes críticos, acompanhada de mecanismos para reduzir o risco de starvation. Também é necessário validar a sincronização dos dispositivos com testes concorrentes.

A escolha do algoritmo deve considerar diferentes objetivos. SJF e Prioridade apresentaram menores médias de espera neste cenário, enquanto Round Robin apresentou menor tempo médio de resposta. Na memória, Optimal obteve a menor taxa de faltas, seguido por LRU e FIFO.

7. Conclusão

O desenvolvimento do KernelLab permitiu comparar diferentes estratégias de escalonamento da CPU, substituição de páginas e controle de acesso a dispositivos. Os resultados demonstram que cada algoritmo possui vantagens e limitações, e que a escolha depende das necessidades do sistema.

No cenário da MedControl Systems, o atendimento de processos críticos merece atenção especial, assim como a prevenção de starvation e a segurança no acesso aos dispositivos. Os resultados também mostram a importância de testar diferentes cenários antes de definir uma estratégia como a mais adequada.

Como melhorias futuras, propõe-se implementar e testar o escalonamento prioritário preemptivo, demonstrar a concorrência real entre threads e ampliar os testes automatizados. Dessa forma, o projeto poderá representar de maneira mais completa os conceitos estudados na disciplina.

8. Referências

1. TANENBAUM, Andrew S.; BOS, Herbert. Sistemas operacionais modernos. 4. ed. São Paulo: Pearson, 2016.
2. UNIVERSIDADE FEDERAL DO VALE DO SÃO FRANCISCO (UNIVASF). Disponível em: https://www.docentes.univasf.edu.br/max.santana/material/OAC-II/sistemas_multiprogramados.pdf. Acesso em: 15 out. 2026.
3. TECNOBLOG. Disponível em: https://tecnoblog.net/responde/o-que-e-um-sistema-operacional/. Acesso em: 09 out. 2026.
4. MEDCONTROL SYSTEMS. Material fornecido no enunciado da atividade de Sistemas Operacionais.
5. PYTHON SOFTWARE FOUNDATION. Disponível em: https://docs.python.org/3/. Acesso em: 10 out. 2026.

6. MATPLOTLIB DEVELOPMENT TEAM. Disponível em: https://matplotlib.org/stable/. Acesso em: 10 out. 2026.